

Math & Comparison Instructions

ADD | SUB | MUL | DIV | CMP | Analogue Scaling | Alarm Logic

What You Will Learn

- ADD, SUB, MUL, DIV, MOD — arithmetic with real PLC examples
- EQU, NEQ, LES, GRT, GEQ — comparison instructions for control decisions
- Analogue scaling: converting 4-20 mA to engineering units step by step
- Building alarm logic and production counters with math instructions

Analogue Scaling Formula

$$\begin{aligned} \text{EU} &= (\text{Raw} / 27648) \\ &\times (\text{EU_max} - \text{EU_min}) \\ &+ \text{EU_min} \end{aligned}$$

Awet G. Nway

Founder, AYE Tech Hub | PLC & Industrial Automation Engineer

PLC-007: Math & Comparison Instructions

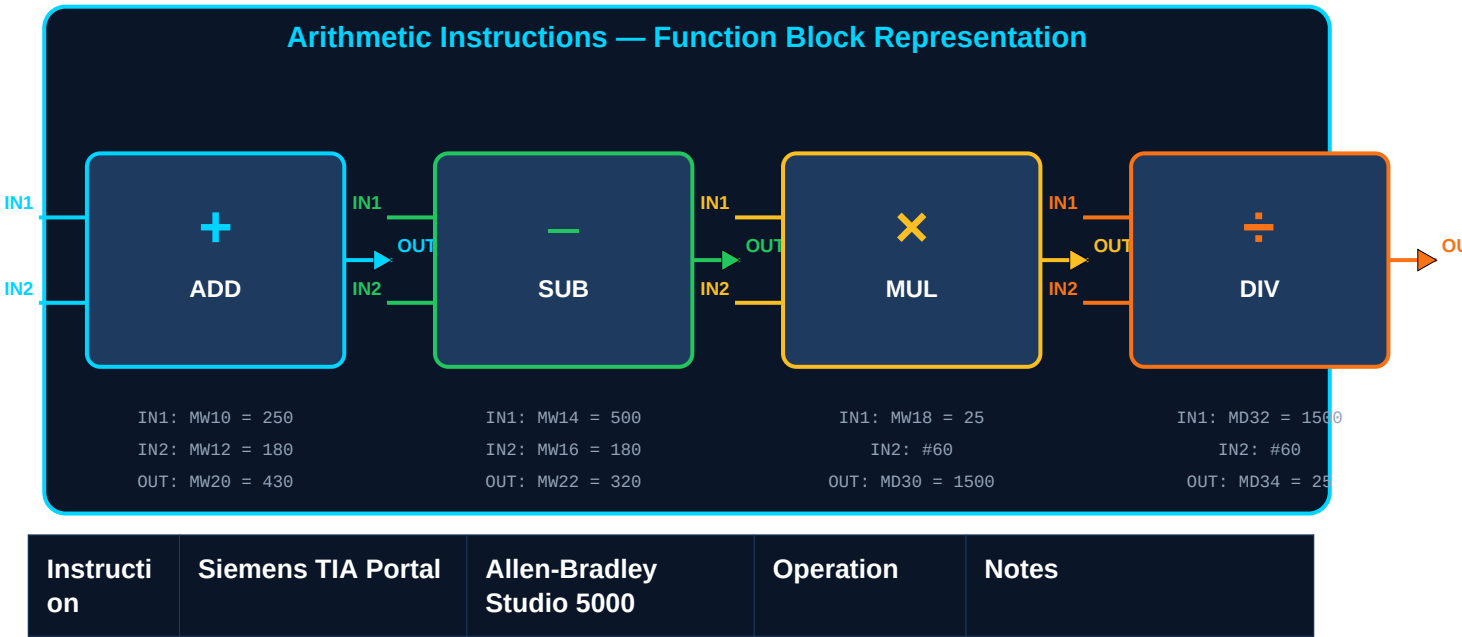
A PLC that can only switch outputs ON and OFF is limited to simple sequencing. The real power of a modern PLC lies in its ability to perform calculations — counting production, calculating flow rates, scaling sensor signals, detecting alarms, and adjusting setpoints dynamically. Lesson PLC-007 covers the complete set of arithmetic and comparison instructions that transform a PLC from a relay replacement into a true process controller.

Learning Objectives

- Apply ADD, SUB, MUL, DIV, and MOD arithmetic instructions with correct data types
- Use mathematical functions: SQRT, ABS, EXPT, and LN/LOG for engineering calculations
- Write comparison instructions (EQU, NEQ, LES, LEQ, GRT, GEQ) to drive alarm and control logic
- Scale analogue input signals from raw PLC integers to real engineering units using math instructions
- Build production rate calculators, alarm systems, and setpoint comparisons using combined math and comparison logic

1. Arithmetic Instructions

Arithmetic instructions perform mathematical operations on integer or real values stored in PLC memory. Each instruction reads one or two source values, performs the calculation, and writes the result to a destination register. The enable (EN) input must be TRUE for the instruction to execute.



ADD	ADD(IN1, IN2, OUT) or + operator in ST	ADD tag1, tag2, dest	OUT = IN1 + IN2	Use INT for counters; REAL for measurements. Overflow if INT result > 32767.
SUB	SUB(IN1, IN2, OUT) or – in ST	SUB tag1, tag2, dest	OUT = IN1 – IN2	Result can be negative. Use DINT if result may exceed 32767.
MUL	MUL(IN1, IN2, OUT) or * in ST	MUL tag1, tag2, dest	OUT = IN1 × IN2	CRITICAL: INT × INT result can easily overflow. Use DINT output or REAL inputs for large products.
DIV	DIV(IN1, IN2, OUT) or / in ST	DIV tag1, tag2, dest	OUT = IN1 ÷ IN2	Integer division truncates: 7 ÷ 2 = 3 (not 3.5). Use REAL data type for decimal results.
MOD	MOD(IN1, IN2, OUT)	MOD tag1, tag2, dest	OUT = IN1 mod IN2 (remainder)	Returns remainder after division: 7 MOD 2 = 1. Useful for cyclical counting and recipe stepping.

Mathematical Functions

Function	Operation	PLC Use Case	Data Type
SQRT	Square root of input	Flow calculation: $Q = k \times \text{SQRT}(\Delta P)$ for differential pressure flowmeters	REAL only
ABS	Absolute value (removes sign)	Error magnitude: $\text{ABS}(\text{Setpoint} - \text{Actual})$ for PID error calculation	INT or REAL
EXPT	IN1 raised to power IN2	Power law calculations in process engineering and heat transfer formulas	REAL only
LN	Natural logarithm (base e)	pH calculation, reaction kinetics, exponential decay modelling	REAL only
SIN / COS / TAN	Trigonometric functions	Cam profile generation, robot arm kinematics, angle calculations	REAL (radians)

2. Comparison Instructions

Comparison instructions compare two values and output a Boolean result (TRUE or FALSE). This Boolean output can be used like any other contact in ladder logic — to enable coils, trigger alarms, change operating modes, or branch between different control sequences. Comparison instructions are the foundation of all alarm logic, setpoint checking, and conditional process control.

Comparison Instructions — Six Types with Boolean Result

EQU
==
Equal

BOOL

NEQ
!=
Not Equal

BOOL

LES
<
Less Than

BOOL

LEQ
<=
Less or Equal

BOOL

GRT
>
Greater Than

BOOL

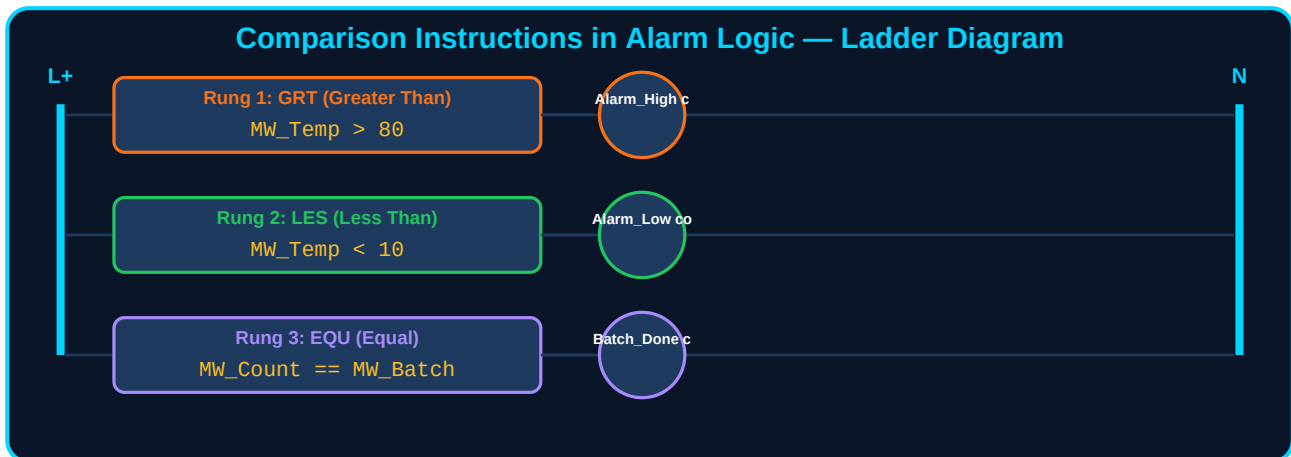
GEQ
>=
Greater or Equal

BOOL

All comparison instructions output a BOOL (TRUE/FALSE). Use this output to drive coils, branch logic, or trigger alarms.

Instruction	Meaning	TIA Portal ST	Result = TRUE when...	Engineering Example
EQU (==)	Equal to	MW10 = MW12	Source A exactly equals Source B	Recipe step complete: Step_Counter == Step_Target
NEQ (!=)	Not equal to	MW10 <> MW12	Source A is different from Source B	Fault active: Fault_Code <> 0
LES (<)	Less than	MW10 < 100	Source A is strictly less than Source B	Low level alarm: Tank_Level < 10
LEQ (<=)	Less than or equal	MW10 <= 100	Source A is less than or equal to B	Soft low setpoint with hysteresis
GRT (>)	Greater than	MW10 > 80	Source A is strictly greater than B	High temperature alarm: Temp_PV > 80.0
GEQ (>=)	Greater or equal	MW10 >= 80	Source A is greater than or equal to B	Pump start permissive: Tank_Level >= 20

Alarm Logic Using Comparison Instructions

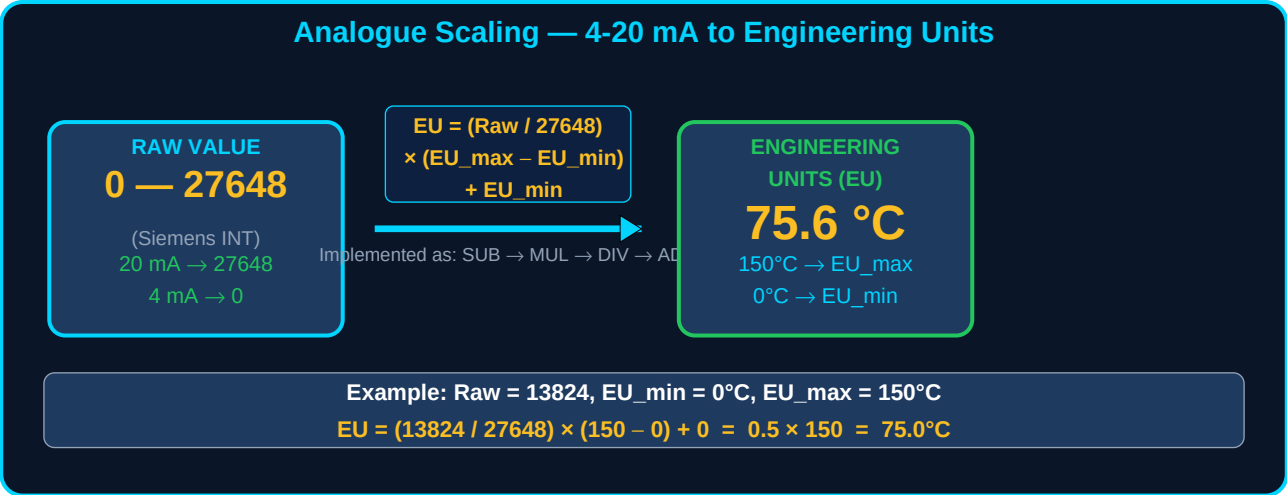


The comparison instruction outputs a BOOL. This BOOL can be wired as a contact to branch logic further, or directly as an enable to an output coil for a simple alarm. Comparison instructions replace dozens of traditional relay timer and setpoint comparator devices with a single program rung.

3. Analogue Scaling — The Most Critical Math Application

Every PLC receives analogue inputs (temperature, pressure, flow, level) as raw integer values — not as real-world engineering units. A 4-20 mA temperature transmitter connected to a Siemens analogue input card produces values from 0 to 27,648. The PLC program must convert this to the actual temperature in °C using a scaling calculation built from arithmetic instructions.

Siemens uses 0–27,648 for 0–20 mA (4 mA minimum = 0, 20 mA = 27,648). Allen-Bradley uses 0–32,767 for 4–20 mA (or 0–20 mA depending on the module configuration). Always check the module datasheet for your specific platform.



Implementing Scaling in PLC Structured Text

S t e p	Operation	Siemens ST Code	Result with Raw=13824
1	Cast raw integer to REAL to allow decimal division	Raw_Real := INT_TO_REAL(AI_Raw);	Raw_Real = 13824.0
2	Divide by maximum raw count (27648)	Fraction := Raw_Real / 27648.0;	Fraction = 0.5
3	Calculate engineering unit span	Span := EU_Max - EU_Min;	Span = 150.0 – 0.0 = 150.0
4	Multiply fraction by span	Scaled := Fraction * Span;	Scaled = 0.5 × 150.0 = 75.0
5	Add minimum engineering unit offset	EU_Out := Scaled + EU_Min;	EU_Out = 75.0 + 0.0 = 75.0°C

Scaling in a Single Formula (Structured Text)

```
Temperature_C := (INT_TO_REAL(AI_Raw) / 27648.0) × (T_Max – T_Min) +  
T_Min;
```

This single ST line implements the complete scaling. Replace 27648.0 with 32767.0 for Allen-Bradley systems. The SCALE instruction in some platforms (Siemens FC105 / AB SCL) does this automatically — but understanding the underlying math lets you implement scaling on any platform without depending on library functions.

Reverse Scaling — Engineering Units to Raw Output

When driving an analogue output (e.g. sending a 4–20 mA setpoint to a VFD or control valve), you need to reverse-scale: convert your engineering unit setpoint back to a raw integer for the analogue output card.

```
AO_Raw := INT((EU_Setpoint – EU_Min) / (EU_Max – EU_Min) × 27648.0);
```

4. Real-World Applications of Math Instructions

Math and comparison instructions combine to create the control logic that drives real industrial processes. Here are four of the most common applications engineers encounter on the job.

Application 1 — Production Rate Calculation

S t e p	Description	PLC Logic
1	Count products using a photoelectric sensor on a counter instruction	CTU: PB_Sensor → Part_Count (increments each product)
2	Capture count at each time interval (e.g. every 60 seconds using TON timer)	On timer done: Rate_Count := Part_Count – Prev_Count; Prev_Count := Part_Count
3	Calculate parts per minute rate	Parts_Per_Min := Rate_Count; (already parts per 60 sec = per min if timer = 60s)
4	Compare rate to target for alarm	IF Parts_Per_Min < Target_Rate THEN Rate_Low_Alarm := TRUE; END_IF
5	Reset and accumulate for shift total	Shift_Total := Shift_Total + Rate_Count; (runs each timer cycle)

Application 2 — Temperature Alarm with Hysteresis

A simple high-temperature alarm using only a GRT comparison trips and resets rapidly if the temperature oscillates near the alarm setpoint. This is called "chattering." Hysteresis prevents chattering by using different setpoints for alarm ON and alarm RESET.

Logic	Condition	Action	Example (setpoint 80°C, hysteresis 3°C)
Alarm ON rung	Temp_PV GRT 80.0 (high alarm setpoint)	SET Alarm_High bit to TRUE (latching)	Alarm triggers when temperature rises above 80°C
Alarm RESET rung	Temp_PV LEQ 77.0 (setpoint minus hysteresis)	RESET Alarm_High bit to FALSE	Alarm clears only when temperature falls below 77°C
Net effect	Temperature oscillating 78–81°C	Alarm stays active — no chattering	Without hysteresis, alarm would toggle continuously in this range

Application 3 — Flow Calculation from Differential Pressure

Principle: $\text{Flow} = k \times \text{SQRT}(\Delta P)$. Differential pressure transmitters are the most common industrial flow measurement method. The PLC reads the differential pressure in engineering units, then calculates actual flow using the SQRT instruction.

Instruction	Operation	Values
AI scaling	Scale AI_DP_Raw to differential pressure in kPa	$\text{DP_kPa} := \text{scale}(\text{AI_DP_Raw}, 0, 100.0); \rightarrow 45.0 \text{ kPa}$
SQRT	Calculate square root of differential pressure	$\text{DP_Sqrt} := \text{SQRT}(\text{DP_kPa}); \rightarrow \text{SQRT}(45.0) = 6.708$
MUL	Multiply by flow coefficient K (from flow meter datasheet)	$\text{Flow_m3h} := \text{DP_Sqrt} \times \text{K_Factor}; \rightarrow 6.708 \times 14.9 = 99.95 \text{ m}^3/\text{h}$
Compare	Check flow against minimum and maximum alarms	$\text{IF Flow_m3h} < 20.0 \text{ THEN Flow_Low} := \text{TRUE}; \text{END_IF}$

5. Practice Exercises

#	Exercise	Instructions
1	Basic arithmetic chain	In Siemens TIA Portal or AB Studio 5000: MW10 = 150, MW12 = 75. Write rungs to calculate and store: sum in MW20, difference in MW22, product in MD24 (DINT), quotient in MW26. Verify results in the watch table.
2	Temperature alarm with hysteresis	Simulate temperature in MW_Temp. Write a rung that SETs Alarm_High when MW_Temp > 85 and RESETs it when MW_Temp < 82. Test by incrementing the simulated temperature and observing alarm behaviour.
3	Analogue scaling	Simulate a raw analogue value MW_AI_Raw = 13824. Write Structured Text to scale it to °C assuming EU_Min = 0°C, EU_Max = 150°C, Max_Raw = 27648. Verify: result should be 75.0°C.
4	Production counter with rate	Create a Part_Count counter. Add a 60-second TON timer. On timer done: calculate Parts_Per_Min = Part_Count - Prev_Count. Store Prev_Count = Part_Count. Compare Parts_Per_Min to 50 and set Rate_Low alarm if below.
5	Reverse scaling for AO	Speed setpoint = 35.0 Hz, EU_Min = 0 Hz, EU_Max = 50 Hz. Write ST code to calculate the AO raw value (0–27648 scale). What is the expected AO_Raw output? Verify: $(35.0/50.0) \times 27648 = 19,353.6 \rightarrow 19354$.

Lesson Summary

Instruction / Concept	Key Rule
ADD / SUB / MUL / DIV	Match data types: INT × INT can overflow → use DINT or REAL for products. DIV on integers truncates to whole number.
MOD	Returns remainder after integer division. MOD 10 creates a 0-9 counter useful for recipe cycling and batch sequencing.
SQRT	REAL inputs and output only. Input must be positive (add ABS if signal may go negative). Used for differential pressure flow calculation.
EQU / NEQ	EQU for exact matches (step counters, recipe selection). NEQ to detect any active fault (Fault_Code NEQ 0).
LES / LEQ / GRT / GEQ	Use GRT for high alarms, LES for low alarms. Add hysteresis (separate ON/RESET setpoints) to prevent alarm chattering.
Analogue scaling	Formula: $EU = (Raw / Max_Raw) \times Span + EU_Min$. Convert raw INT to REAL first to avoid integer truncation errors.
Reverse scaling	Formula: $AO_Raw = (EU_Value - EU_Min) / Span \times Max_Raw$. Always clamp AO_Raw to 0–27648 before writing to output module.

What Is Next — PLC-008

PLC-008: Function Block Diagram (FBD) — The graphical programming language where logic is represented as connected signal-flow blocks. FBD is ideal for process control, PID loops, and analogue signal processing. Covers standard function blocks (AND, OR, NOT, SR, RS), user-defined function blocks (FBs), and when to choose FBD over Ladder Diagram.

Resource	Details
PDF Series	PLC-001 through PLC-030 — ayetechub.com/pdfs.html
Free PLC Simulator	Ladder Logic Simulator — ayetechub.com/downloads
Community	Telegram: t.me/ayetechub